



Enemies: Controller, Stats, Type, Movement, AnimationController



Информация о врагах, типах, их функционале и работе.

EnemyType (variant).



Каждый prefab врага имеет EnemyType (EnemyStats.cs содержит такое поле) поле: 15 типов врагов.

P.s. возможно правильное было назвать перечисление: EnemyVariant.

```
1 public enum EnemyType
2 {
3     Rot,
4     Wolf,
5     Slime_Green,
6     Slime_Blue,
7     Slime_Fire,
8     Plant1,
9     Plant2,
10    Plant3,
11    Goblin1,
12    Goblin2,
13    Goblin3,
14    Orc,
15    Mage,
16    HorseRider,
17    BeholderBoss
18 }
```

Благодаря полю и явному разделению врагов по типу(варианту) можно выстроить логику, которая будет выполнять те или иные действия по отношению к определенному типу. В момент написания документа идёт работа над EnemyObjectPool, чтобы пул мог выдавать объекты врагов через запрос по его типу.

EnemyRoleTier.



Перечисление для определения роли врага. Было добавлено задолго после создания врагов, когда дело дошло до реализации генератора волн.

```
1 public enum EnemyRoleTier
2 {
3     Boss,
4
5     Fast_1,
6     Fast_2,
7     Fast_3,
8
9     Tank_1,
10    Tank_2,
11    Tank_3,
```

```

12
13     Swarm_1,
14     Swarm_2,
15     Swarm_3
16 }

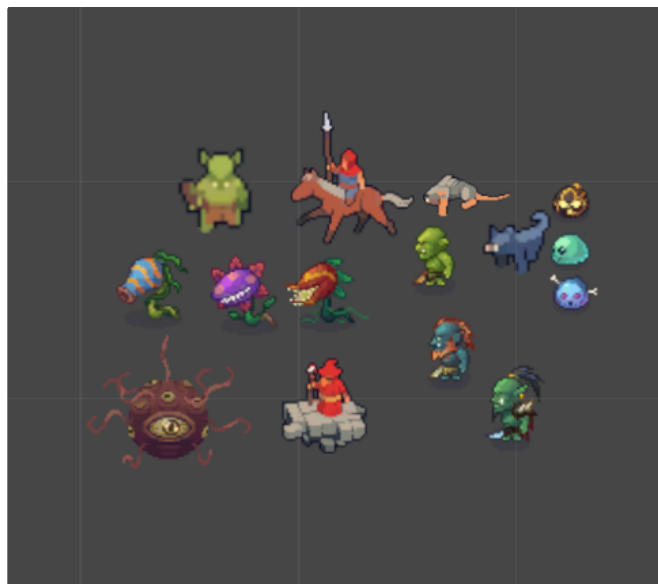
```

Enemies Stats.

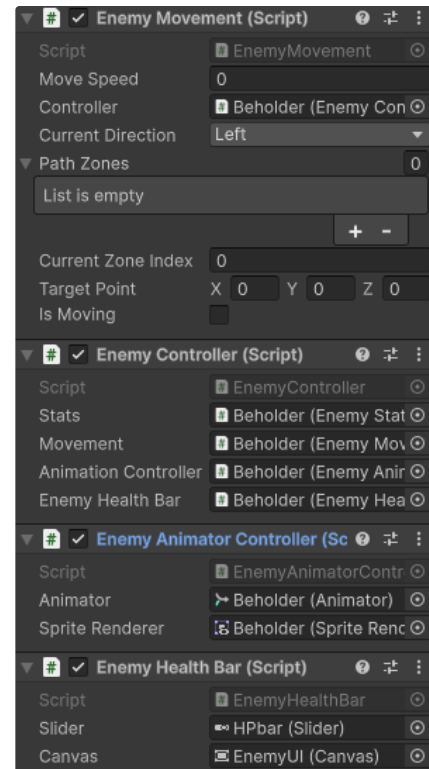
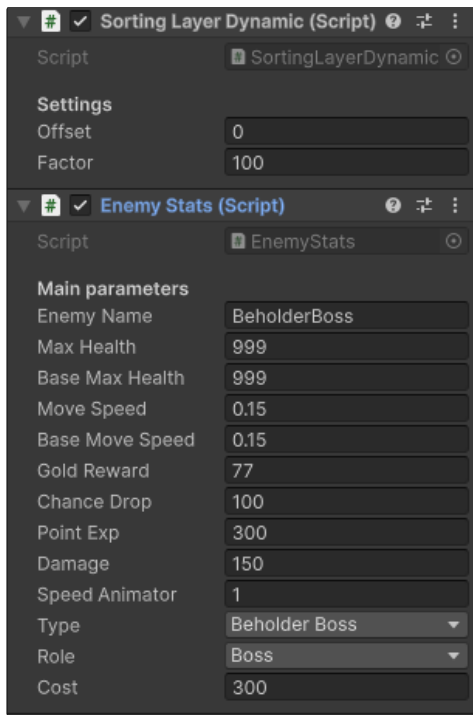
i Характеристики врагов (стартовые, без множителей):

Враг	HP	Speed	Money Drop	Rate Drop	Exp (hp*sp+ Damage)	Damage	Cost	Enemy Role
Слайм	40	0.30	5	10%	17	5	8	swarm
Слайм синий	65	0.40	8	15%	36	10	12	swarm
Слайм огненный	110	0.35	15	25%	43	15	18	tank_1
Крыса	80	0.5	20	20%	50	10	14	fast_1
Волк	110	0.7	25	30%	92	15	22	fast_2
Гоблин 1	100	0.4	15	30%	55	15	18	tank_1
Гоблин 2	120	0.55	25	40%	86	20	24	tank_2
Гоблин 3	130	0.65	35	50%	105	30	32	tank_3
Растение 1	180	0.3	20	20%	74	20	20	Tank_1
Растение 2	210	0.4	25	25%	109	25	28	Tank_2
Растение 3	270	0.35	30	50%	125	30	36	Tank_3

Маг на камне	360	0.3	40	70%	158	50	45	Tank_3
Всадник	140	1	45	70%	170	30	40	Fast_3
Орк	460	0.25	50	70%	185	70	60	Tank_3
Бихолдер	999	0.15	Artefact	100%	300	150	300	Boss



Отображение всех мобов текущей версии.



EnemyStats.

```

1 using System;
2 using UnityEngine;
3
4 public class EnemyStats : MonoBehaviour
5 {
6     [Header("Main parameters")]
7     public string enemyName;
8     public float maxHealth = 100f;
9     public float baseMaxHealth = 50f;
10    public float moveSpeed = 2f;
11    public float baseMoveSpeed = 0.5f;
12    public int goldReward = 0;
13    public int chanceDrop = 0;
14    public int pointExp = 0;
15    public float damage = 10f;
16    public float speedAnimator = 1f;
17    public EnemyType type;
18    public EnemyRole role;
19    public int cost = 0;
20
21    [HideInInspector] public float currentHealth;
22    [HideInInspector] public int difficultyLevel = 1;
23
24    public event Action OnDeathAction;
25
26
27    private void Awake()
28    {
29        ResetStats();
30    }
31
32    public void TakeDamage(float amount)
33    {

```

```

34     currentHealth -= amount;
35     if (currentHealth <= 0)
36     {
37         currentHealth = 0;
38         OnDeathAction?.Invoke();
39     }
40 }
41
42 public float GetDamage() => damage;
43 public bool IsDead() => currentHealth <= 0;
44
45 public void ResetStats()
46 {
47     maxHealth = baseMaxHealth * DifficultyManager.Instance.HpMultip
48     currentHealth = maxHealth;
49     moveSpeed = baseMoveSpeed * DifficultyManager.Instance.SpeedMul
50     pointExp = Convert.ToInt32(maxHealth * moveSpeed + damage);
51 }
52 }
53 // Скрипт со всеми характеристиками врага

```

EnemyController.

```

1 using System;
2 using System.Collections;
3 using UnityEngine;
4 public class EnemyController : MonoBehaviour, IPoolIdentity
5 {
6     [SerializeField] private EnemyStats stats;
7     [SerializeField] private EnemyMovement movement;
8     [SerializeField] private EnemyAnimatorController animationControll
9     [SerializeField] private EnemyHealthBar enemyHealthBar;
10
11     public int WaveIndex { get; private set; }
12
13     public EnemyType Type => stats.type;
14
15     public event Action<float> OnSpeedChanged;
16
17     private int roll;
18
19     private void Awake()
20     {
21         stats = GetComponent<EnemyStats>();
22         movement = GetComponent<EnemyMovement>();
23         animationController = GetComponent<EnemyAnimatorController>();
24         enemyHealthBar = GetComponent<EnemyHealthBar>();
25
26         if (stats == null || movement == null || animationController ==
27             || enemyHealthBar == null)
28         {
29             Debug.LogWarning("Enemy has not: EnemyStats | EnemyMovemen
30         }
31
32         movement.OnFinishedPath += ReachBase;
33     }
34
35     private void OnEnable()
36     {
37         stats.ResetStats();
38         movement.ResetMovement();
39         animationController.ResetAnimation();
40         enemyHealthBar.Init(stats.maxHealth);
41         enemyHealthBar.Show();
42     }

```

```

43         stats.OnDeathAction += Die;
44         animationController.OnDeathAnimationFinished += ReturnToPool;
45     }
46
47     private void OnDisable()
48     {
49         stats.OnDeathAction -= Die;
50         animationController.OnDeathAnimationFinished -= ReturnToPool;
51     }
52
53     public void Die()
54     {
55         movement.StopMovement();
56         animationController.PlayDeath(movement.GetCurrentDirection());
57         enemyHealthBar.Hide();
58         GameManager.Instance.AddScore(stats.pointExp);
59         GameManager.Instance.AddKill();
60         CoinDropAttempt();
61
62         EnemySpawnerManager.Instance.NotifyEnemyKilled(WaveIndex);
63     }
64
65     public void ReachBase()
66     {
67         movement.StopMovement();
68         enemyHealthBar.Hide();
69         ReturnToPool();
70         BaseManager.Instance.TakeDamage(stats.GetDamage());
71         if (BaseManager.Instance.CurrentHp > 0)
72         {
73             EnemySpawnerManager.Instance.NotifyEnemyKilled(WaveIndex);
74         }
75     }
76
77
78     private void ReturnToPool()
79     {
80         PoolManager.Instance.Return(this.gameObject);
81     }
82
83     public float GetMoveSpeed() => stats.moveSpeed;
84
85     public void ChangeSpeed(float newSpeed)
86     {
87         OnSpeedChanged?.Invoke(newSpeed);
88         if (stats.moveSpeed != newSpeed)
89         {
90             stats.moveSpeed = newSpeed;
91         }
92     }
93
94     public void TakeDamage(float damage)
95     {
96         stats.TakeDamage(damage);
97         enemyHealthBar.UpdateHealth(damage);
98         animationController.PlayHitFlash();
99     }
100
101     public void PlayWalk(EnemyDirection direction)
102     {
103         animationController.PlayWalk(direction);
104     }
105
106     public float GetDamage()
107     {
108         return stats.GetDamage();
109     }

```

```

110
111     public string GetPoolId() => Type.ToString();
112
113     public void CoinDropAttempt()
114     {
115         roll = UnityEngine.Random.Range(0, 101);
116         if (roll >= stats.chanceDrop)
117         {
118             return;
119         }
120
121         WalletManager.Instance.Add(stats.goldReward);
122     }
123
124     public void SetWaveIndex(int waveIndex)
125     {
126         WaveIndex = waveIndex;
127     }
128 }

```

EnemyAnimatorController.

```

1  using System;
2  using System.Collections;
3  using UnityEngine;
4
5  public enum EnemyAnimation
6  {
7      Walk_Side, Walk_Up, Walk_Down, Death_Side, Death_Up, Death_Down
8  }
9
10 public class EnemyAnimatorController : MonoBehaviour
11 {
12     [SerializeField] private Animator animator;
13     [SerializeField] private SpriteRenderer spriteRenderer;
14     private EnemyAnimation currentAnimation;
15     private bool rightFlip = false;
16
17     private Color baseColor;
18     private Color hitFlash = new Color(0.7f, 0.7f, 0.7f, 0.8f);
19     private float hitFlashDuration = 0.3f;
20     private Coroutine hitFlashCoroutine;
21
22     private float deathFadeDelay = 5f;
23     private float fadeDuration = 1.5f;
24     private Coroutine fadeOutCoroutine;
25
26     public event Action OnDeathAnimationFinished;
27
28
29     private void OnEnable()
30     {
31         ReturnBaseColor();
32         SpeedGameManager.Instance.OnSpeedGameChanged += AnimatorSpeedA
33     }
34     private void Awake()
35     {
36         if (animator == null) { animator = GetComponent<Animator>(); }
37         if (spriteRenderer == null) { spriteRenderer = GetComponent<Sp
38             baseColor = spriteRenderer.color;
39     }
40
41     private void PlayAnimation()
42     {
43         if (animator == null) return;

```

```

44         animator.Play(currentAnimation.ToString());
45         HandleFlip();
46     }
47
48     private void HandleFlip()
49     {
50         if (spriteRenderer == null) return;
51         if (currentAnimation != EnemyAnimation.Death_Side &&
52             currentAnimation != EnemyAnimation.Walk_Side)
53         {
54             spriteRenderer.flipX = false;
55             return;
56         }
57
58         spriteRenderer.flipX = rightFlip;
59     }
60
61     public void PlayWalk(EnemyDirection direction, float speedMultipli
62     {
63         rightFlip = false;
64
65         switch (direction)
66         {
67             case EnemyDirection.Up: currentAnimation = EnemyAnimation.
68             case EnemyDirection.Down: currentAnimation = EnemyAnimatio
69             case EnemyDirection.Left: currentAnimation = EnemyAnimatio
70             case EnemyDirection.Right: currentAnimation = EnemyAnimati
71                 rightFlip = true; break;
72             default: currentAnimation = EnemyAnimation.Walk_Side; bre
73         }
74
75         PlayAnimation();
76         animator.speed = speedMultiplier * SpeedGameManager.Instance.S
77     }
78
79     public void PlayDeath(EnemyDirection direction)
80     {
81         rightFlip = false;
82         switch (direction)
83         {
84             case EnemyDirection.Up: currentAnimation = EnemyAnimation.
85             case EnemyDirection.Down: currentAnimation = EnemyAnimatio
86             case EnemyDirection.Left: currentAnimation = EnemyAnimatio
87             case EnemyDirection.Right: currentAnimation = EnemyAnimati
88                 rightFlip = true; break;
89             default: currentAnimation = EnemyAnimation.Death_Side; bre
90         }
91
92         PlayAnimation();
93         FadeOut();
94         animator.speed = 1f * SpeedGameManager.Instance.SpeedMultiplie
95
96         if (hitFlashCoroutine != null)
97         {
98             StopCoroutine(hitFlashCoroutine);
99             hitFlashCoroutine = null;
100         }
101
102         spriteRenderer.color = baseColor;
103
104     }
105
106     public void ResetAnimation()
107     {
108         rightFlip = false;
109         currentAnimation = EnemyAnimation.Walk_Side;
110

```



```

111         animator.speed = 1f * SpeedGameManager.Instance.SpeedMultiplie
112     }
113
114     private void AnimatorSpeedAdapter(GameSpeed speedType)
115     {
116         float newSpeed = speedType switch
117         {
118             GameSpeed.Pause => 0f,
119             GameSpeed.X1 => 1f,
120             GameSpeed.X2 => 2f,
121             GameSpeed.X4 => 4f,
122             _ => 1f
123         };
124
125         SetAnimatorSpeed(newSpeed);
126     }
127
128     private void SetAnimatorSpeed(float newSpeed)
129     {
130         animator.speed = newSpeed;
131     }
132
133
134     public void PlayHitFlash()
135     {
136         if (fadeOutCoroutine != null) return;
137         if (hitFlashCoroutine != null)
138         {
139             StopCoroutine(hitFlashCoroutine);
140         }
141         hitFlashCoroutine = StartCoroutine(HitFlashRoutine());
142     }
143
144     private IEnumerator HitFlashRoutine()
145     {
146         spriteRenderer.color = hitFlash;
147         yield return new WaitForSeconds(hitFlashDuration);
148         spriteRenderer.color = baseColor;
149         hitFlashCoroutine = null;
150     }
151
152     public void FadeOut()
153     {
154         if (fadeOutCoroutine != null)
155         {
156             StopCoroutine(fadeOutCoroutine);
157         } fadeOutCoroutine = StartCoroutine(DeathFadeRoutine());
158     }
159
160     private IEnumerator DeathFadeRoutine()
161     {
162         yield return new WaitForSeconds(deathFadeDelay);
163         float time = 0f;
164         Color color = spriteRenderer.color;
165
166         while (time < fadeDuration)
167         {
168             time += Time.deltaTime;
169             color.a = Mathf.Lerp(1f, 0f, time / fadeDuration);
170             spriteRenderer.color = color;
171             yield return null;
172         }
173
174         color.a = 0f;
175         spriteRenderer.color = color;
176
177         fadeOutCoroutine = null;

```

```

178         OnDeathAnimationFinished?.Invoke();
179     }
180
181     public void ReturnBaseColor()
182     {
183         spriteRenderer.color = baseColor;
184     }
185
186     private void OnDisable()
187     {
188         SpeedGameManager.Instance.OnSpeedGameChanged -= AnimatorSpeedA
189     }
190 }
191
192 // Скрипт для управления анимациями

```

EnemyMovement.

```

1  using System;
2  using Unity.Mathematics;
3  using UnityEngine;
4  using Random = UnityEngine.Random;
5
6  public enum EnemyDirection
7  {
8      Up, Down, Left, Right
9  }
10 public class EnemyMovement : MonoBehaviour
11 {
12     [SerializeField] private float moveSpeed;
13     [SerializeField] private EnemyController controller;
14     [SerializeField] private EnemyDirection currentDirection = EnemyDi
15     [SerializeField] private Transform[] pathZones;
16     [SerializeField] private int currentZoneIndex = 0;
17     [SerializeField] private Vector3 targetPoint;
18     [SerializeField] private bool isMoving = false;
19
20     public Action OnFinishedPath;
21
22     public EnemyDirection GetCurrentDirection() => currentDirection;
23
24
25     private void OnEnable()
26     {
27         controller.OnSpeedChanged += UpdateMoveSpeed;
28     }
29
30     private void Awake()
31     {
32         controller = GetComponent<EnemyController>();
33         if (controller == null) Debug.LogError("No Enemy controller!!!
34
35         moveSpeed = controller.GetMoveSpeed();
36     }
37
38     private void Start()
39     {
40
41         if (pathZones.Length > 0)
42         {
43             SetNextTarget();
44         }
45     }
46
47     private void Update()

```

```

48     {
49         if (!isMoving || pathZones?.Length == 0) return;
50         MoveTowardsTarget();
51     }
52
53     private void MoveTowardsTarget()
54     {
55         Vector3 direction = (targetPoint - transform.position).normalized;
56         UpdateAnimations(direction);
57
58         transform.position = Vector3.MoveTowards(transform.position,
59             targetPoint, moveSpeed * Time.deltaTime * SpeedGameManager.Instance);
60
61         if (Vector3.Distance(transform.position, targetPoint) < 0.05f)
62         {
63             currentZoneIndex++;
64             if (currentZoneIndex >= pathZones.Length)
65             {
66                 StopMovement();
67
68                 OnFinishedPath?.Invoke();
69
70                 return;
71             }
72             SetNextTarget();
73         }
74     }
75
76     private void SetNextTarget()
77     {
78         var zone = pathZones[currentZoneIndex];
79         var collider = zone.GetComponent<Collider2D>();
80         if (collider != null)
81         {
82             var bounds = collider.bounds;
83             float x = Random.Range(bounds.min.x, bounds.max.x);
84             float y = Random.Range(bounds.min.y, bounds.max.y);
85             targetPoint = new Vector3(x, y, transform.position.z);
86         }
87         if (collider == null)
88         {
89             targetPoint = zone.position;
90         }
91     }
92
93     public void SetPath(Transform[] path)
94     {
95         pathZones = path;
96         currentZoneIndex = 0;
97         if (path.Length > 0)
98         {
99             SetNextTarget();
100         }
101     }
102
103     private void UpdateAnimations(Vector3 direction)
104     {
105         if (Mathf.Abs(direction.x) > Mathf.Abs(direction.y))
106         {
107             currentDirection = direction.x > 0 ? EnemyDirection.Right : EnemyDirection.Left;
108         }
109         else
110         {
111             currentDirection = direction.y > 0 ? EnemyDirection.Up : EnemyDirection.Down;
112         }
113
114         controller.PlayWalk(currentDirection);

```

```
115     }
116
117     public void StopMovement()
118     {
119         isMoving = false;
120     }
121
122     public void StartMovement()
123     {
124         isMoving = true;
125     }
126
127     private void UpdateMoveSpeed(float newSpeed)
128     {
129         moveSpeed = newSpeed;
130     }
131
132     public void ResetMovement()
133     {
134         currentDirection = EnemyDirection.Left;
135         moveSpeed = controller.GetMoveSpeed();
136     }
137
138     private void OnDisable()
139     {
140         controller.OnSpeedChanged -= UpdateMoveSpeed;
141     }
142 }
```